

# WEEKLY PROGRESS REPORT

## Production-Grade SOC Dashboard, Thread-Safe IPC & Secure Active Response Engineering

### 1. Contribution Scope

This week focuses on designing a real-time Security Operations Center (SOC) dashboard utilizing the Streamlit and Altair frameworks, coupled with thread-safe Inter-Process Communication (IPC) via `threading.Lock` from the core engine. Additionally, source code cleaning and secure coding hardening were executed to eliminate dangerous system calls, thereby finalizing the closed-loop, anti-evasion IDPS framework.

### 2. Objectives of This Week

- **Asynchronous Telemetry:** Engineer an asynchronous telemetry data exporter (`export_dashboard_telemetry`) to synchronize network state profiles into an intermediate JSON data buffer.
- **Race Condition Elimination:** Implement a mutual exclusion lock (`threading.Lock`) to completely eliminate memory-corruption race conditions between the main processing loop and the asynchronous host rehabilitation thread.
- **Active Response Hardening:** Harden the active response subsystem by replacing legacy shell invocation methods with secure, decoupled parameter arrays via `subprocess.run` to nullify OS Command Injection vulnerabilities.
- **Layout Geometric Optimization:** Optimize the geometric layout of the SOC dashboard to fix the 90-degree vertical rotation flaw of IP labels and constrain the graphical bar size when testing with a single compromised host.

### 3. Detailed Work

- **Thread-Safe IPC Architecture:** Implemented a global synchronization lock named `state_lock`. When the `export_dashboard_telemetry` function traverses the internal `volume_quota` memory cache to export metrics, it must execute strictly within the `with state_lock: context manager`. This structurally prevents system crashes caused by a `RuntimeError: dictionary changed size during iteration` when the background unblocking thread mutates host keys concurrently.
- **Kernel-Level Secure Code Hardening:** Completely removed vulnerable legacy `os.system()` invocations. All network defense iptables rules were refactored into modern decoupled static arrays. Untrusted input fields such as the source IP (`src_ip`) are strictly bound as independent execution

parameters, ensuring attackers cannot execute reverse shell injections via crafted malicious FQDN subdomains.

- **Graphical Layout and Geometric Corrections:** Migrated the frontend telemetry plotting pipeline from native `st.bar_chart` to the advanced Altair rendering engine. Configured the coordinate property `axis=alt.Axis(labelAngle=0)` to firmly force horizontal, clean text alignment for target IP labels. Concurrently, enforced a strict `size=40` attribute inside the structural bar chart initialization to keep the user interface lean and professional even during single-target simulation testing.

## 4. Results / Output

- **Subsystem Integration:** Simultaneously executed all 3 operational layers: `test.py` (Core Backend Engine), `dashboard.py` (Streamlit UI running over the Network URL at port 8501), and `test.sh` (The automated attacker script).
- **Anti-Evasion Trigger:** As soon as traffic generated by `test.sh` broke past the 1000-byte allocation threshold while sustaining localized temporal anomaly cycles, the correlation brain automatically triggered an Emergency Escalation rule, force-raising the composite risk score directly to **99/100**.
- **Real-Time SOC Synchronization:** The Streamlit SOC interface refreshed dynamically within 0.2 seconds: the Currently Isolated IPs KPI card instantly incremented to 1, the bar chart projected corresponding volume data blocks cleanly, and the active network blocklist populated the quarantined host's IP with a red critical banner. Simultaneously, active troubleshooting terminal pings on the attacker's end failed instantly, displaying a definitive [BLOCKED] network dropping state.